A Tutorial on Upper Bounds for Condorcet Domains

Søren Riis

Review draft, 1 November 2026

This tutorial presents the upper-bound results of Pavlos Moros and Søren Riis [1], following the earlier AI-assisted note by Moros [2]. It develops the proof with worked examples and explicit links to the joint Lean formalisation. The contribution of this tutorial is its exposition, organisation and verification guide; the underlying research results retain their attribution to the original works.

The bounds have general exponential base $32/11$ and peak-pit base $8/3$, together with root-growth limits and a polynomial refinement. The seven final results are stated in Section 9.

Start with the first-pass guide on page 3. It explains the whole route before the technical details. The ordinary prose and worked examples explain why each step should be true; the numbered statements and proofs supply the mathematical argument. The small *Lean correspondence* paragraphs identify actual definitions and declarations, their premises, and any change of notation. They can be skipped on a first reading. Appendix A indexes every declaration exported by the two final entry modules, and every further result in the independent 20-result audit.

The finite part is computer-assisted. The structural and limiting arguments are given in full. Sections 6 and 11 specify the finite inequality, its rational data, exhaustive coverage, checker soundness and exceptional cases. The companion Lean project supplies every literal table and finite proof. A human reader can follow the logic without personally evaluating all table entries; the strongest bound still uses a certificate.

	Idea	What it achieves
1	Delete one alternative	A lower bound on deletion outputs becomes a recurrence
2	Count fibre runs by positions minus edges	The exact identity $\Sigma + G + A = nM$
3	Assign flexibility to distinct obstructions	The gap inequality
4	Decompose using majority projection	General domains reduce to full extreme-law pieces
5	Average over conditioned local windows	A finite inequality controls a long order
6	Add corrections that telescope	Only the two outside ends remain
7	Repeat ordered copies	A bounded outside error disappears
8	Search for weights	Discover a plausible local inequality
9	Generate every admissible constraint system	Prove coverage, including fixed comparisons
10	Redistribute exceptional deficits	Complete the local inequality
11	Freeze and check an exact certificate	Separate exploratory computation from proof
12	Prove the encoding is faithful	The computed permutations represent the intended preference orders

The numerical comparison in Section 10 uses the same values of n to show what successive proof steps achieve. A second table also uses published small-case classifications, including $f(8) = 224$, as explicitly separate starting information. Those classifications are not premises of the $32/11$ proof. The separate six-position hand proof below 3 is outside this tutorial's proof chain.

Contents

A first pass: what the proof has to do **3**

1 The objects being counted **6**

2 Idea 1: deletion turns a structural count into a recurrence **8**

3 Idea 2: runs are positions minus edges **9**

4 Idea 3: assign flexibility to distinct obstructions **10**

5 Idea 4: decompose by majority projection **12**

6 The local inequality needed by Ideas 5 and 6 **14**

7 Ideas 5 and 6: local windows and telescoping corrections **17**

8 Idea 7: repeated copies remove the bounded error **19**

9 The transfer and all seven final conclusions **20**

10 What successive ideas achieve at concrete sizes **23**

11 Ideas 8–12: how the finite input becomes a proof **24**

A Complete definition and entry-point index **27**

B Verification evidence, reproduction and scope **29**

A first pass: what the proof has to do

An upper bound limits the size of *every* Condorcet domain. An explicit construction supplies one domain and hence a lower bound. We will use small constructions to make the counting visible, and then explain why the same counting limits all domains.

1. Erase a label and count what survives

Suppose a domain D has M orders on n alternatives. Erasing a label can merge different orders. The proof asks how much merging is possible, averaged over the n choices of erased label. Here are the four quantities to keep in view:

Symbol	What we count	How to picture it
M	Orders in D	The original collection of rankings.
Σ	Distinct deletion outputs	An output remembers both the deleted label and the remaining order.
G	Separations in insertion positions	Several separated runs of possible positions for one reinserted label contribute extra units.
A	Enabled adjacent swaps	Each swap edge counts twice, once from either endpoint.

An insertion fibre is just the list of places where one erased label could have been. In such a list,

$$\text{number of runs} = \text{occupied places} - \text{edges between occupied neighbours}.$$

Summing this elementary path identity gives $\Sigma + G + A = nM$. This is the counting framework for the whole proof.

Every deletion of a Condorcet domain is still Condorcet. Therefore $\Sigma \leq nf(n-1)$. To bound M, we need a lower bound on Σ: the orders cannot all disappear into very few outputs.

2. Prove two estimates, for different reasons

The structural argument gives, for a maximal domain,

$$\boxed{G + (n+1)M \leq 3\Sigma.} \qquad \text{(gap count)}$$

Its main idea is an injective assignment: each freely movable label forces a distinct obstruction elsewhere. Once that assignment is proved, the inequality follows by counting moves. No finite certificate is used in this step.

The second estimate is

$$\boxed{\Sigma + G \geq \frac{3}{8} nM.} \qquad \text{(component count)}$$

It comes from splitting a full domain into adjacent-swap components. Each component is the full solution set of extreme triple laws and fixed pair comparisons. On these simpler pieces, insertion positions form intervals and the finite argument proves that deletion retains at least a $3/8$ fraction of the nM order/label incidences. Splitting counts separated insertion runs separately, which explains the additional G.

3. The fraction $32/11$ is now forced

The two estimates give

$$\frac{3}{8}nM \le \Sigma + G \le 4\Sigma - (n+1)M.$$

Move the $(n+1)M$ term to the left and multiply by eight:

$$(11n+8)M \le 32\Sigma \le 32nf(n-1).$$

Choose a domain of maximum size $M = f(n)$. This is the advertised recurrence, and its limiting multiplier is $32/11$.

One can also see the arithmetic without large counts. Set $s = \Sigma/(nM)$, $g = G/(nM)$, with $n, M > 0$. The estimates say

$$s + g \ge \frac{3}{8}, \qquad g + 1 + \frac{1}{n} \le 3s.$$

Together they force $s \ge 11/32 + 1/(4n)$. The term $1/(4n)$ accounts exactly for the $+8$ in the recurrence denominator.

> **Lean correspondence.**
>
> This overview restates `CDCert.TwoInputs.first_input`, `second_input` and `transfer`. The normalised variables are an explanatory change of units. The precise unnormalised statements and their hypotheses appear beside the later proofs.

4. Where the computer enters, and why copies help

For the simpler pieces, the difficult input is a particular inequality on seven alternatives. It bounds a weighted swap count by $5/8$ of the family size, plus a correction at each end. The finite certificate proves this inequality for every permitted constraint system.

Fixing the order outside a seven-position window lets us apply that small inequality inside a long order. Consecutive windows share six positions. Their corrections cancel there, leaving a bounded error at the two outside ends. Ordered copies then remove this error: after dividing by the family size, the main counts grow in proportion to the number of copies, while the total number of outside ends remains two. Thus the exact swap estimate is $A \le 5nM/8$. Since interval fibres give $\Sigma = nM - A$, the retained deletion fraction is $3/8$.

This also explains the peak-pit multiplier $8/3$: applying $\Sigma \ge 3nM/8$ to a maximum peak-pit domain and using $\Sigma \le nh(n-1)$ gives $h(n) \le (8/3)h(n-1)$. The general-domain proof transfers this *deletion estimate* through components and the gap count. A geometric estimate for $h(n)$ alone would not supply that structural information.

The twelve ideas as reusable mathematical moves

The following are conceptual analogies, not claims of historical priority. They also distinguish discovery from the eventual proof.

	Move in this proof	General mathematical pattern
1	Count deletion outputs	Double counting followed by induction.
2	Count runs by places minus edges	The vertices-minus-edges formula for a disjoint union of paths.
3	Assign distinct obstructions	An injection, or a charging argument without reuse.
4	Project using majority	Closure under a median operation and projection in Boolean coordinates.
5	Fix the exterior	Conditioning, then interchanging finite sums.
6	Cancel overlap corrections	Telescoping sums; a discrete difference disappears in the interior.
7	Repeat ordered copies	Amplification: divide a bounded error by an arbitrarily large parameter.
8	Search for useful weights	Linear programming and counterexample-guided discovery.
9	Cover every constraint choice	Exhaustive case analysis with inductively proved soundness.
10	Pay exceptional deficits	Redistribution of local surplus, often called discharging.
11	Check exact finite data	Certificate checking by a small trusted verifier.
12	Relate the two representations	A proved bijection preserving the properties and counts used.

Idea 8 discovers a candidate; it is not a premise of the theorem. Ideas 9–11 establish the finite input. Idea 12 ensures that this input really applies to the preference orders in the original question.

1 The objects being counted

For deletion arguments, $n \geq 1$. The extremal functions themselves are also defined at $n = 0$; that initial value is harmless in limits.

Definition 1.1 (Orders, profiles and domains)**.** An order on a finite set X is a strict total order, written as a list from best to worst. A domain is a set D of orders on the same X. For a profile $(\sigma_1, \ldots, \sigma_m)$, a strict majority puts a above b when more than $m/2$ voters do so. A Condorcet domain is a domain for which this majority relation is transitive for every odd positive m and every profile from D. Repeated orders in a profile are allowed. Write $f(n)$ for the maximum possible $|D|$ on n alternatives.

Lean correspondence.

`Condorcet.IsStrictTotal`, `Condorcet.Order`, `Condorcet.Domain`, `Condorcet.majority`, `Condorcet.IsCondorcet`, `Condorcet.f`. An order is a Boolean relation with irreflexivity, transitivity and totality; asymmetry follows. A domain is a `Finset`, so its members are distinct. Profiles are functions $\mathrm{Fin}(m) \to \mathrm{Order}(X)$, with no injectivity requirement. The definition of f takes a finite supremum over exactly these semantic domains. The supplied correspondence source restates these three semantic interfaces in `TutorialCorrespondence.condorcet_definition`, `peak_pit_definition` and `triple_characterization`.

1.1 Triple laws connect voting to combinatorics

On a triple $\{x, y, z\}$, the law xN_i says that x never occupies position i, with $i = 1, 2, 3$ meaning first, middle, last. An extreme law is xN_1 or xN_3.

Definition 1.2 (Peak-pit)**.** A peak-pit domain has a common extreme law on each triple, with the choice of alternative and forbidden position allowed to vary between triples. Let $h(n)$ be its maximum size. No common axis is required.

Lean correspondence.

`Condorcet.Never` uses positions $0, 1, 2$, respectively. `Condorcet.ValueRestricted` means a common never-law per triple; `ExtremeRestricted` requires the position to differ from 1 in this zero-based convention. `IsPeakPit` is explicitly `IsCondorcet` together with `ExtremeRestricted`; `Condorcet.h` takes the corresponding maximum.

Theorem 1.3 (Triple characterisation)**.** *A domain is Condorcet if and only if each triple has a common never-law. In particular, a common extreme law per triple suffices.*

Proof. On a fixed triple, separate the six orders into

$$\{abc, bca, cab\}, \qquad \{acb, cba, bac\}.$$

The three orders in either set, used once each as a three-voter profile, produce a majority cycle. A Condorcet domain must therefore omit at least one order from each set. There are nine choices of the two omitted orders. In every choice, some alternative occupies the same position in both omitted orders; the remaining four orders satisfy that never-law. To see the nine cases without a search, fix the omitted order in the first set to be abc by cyclic relabelling. Pairing it with acb, cba, bac gives respectively aN_1, bN_2, cN_3. Undo the relabelling for the other two first-set choices.

Conversely, suppose all voters satisfy a fixed never-law on a triple. For xN_2, each voter compares x with y, z in the same direction, so the two majority comparisons agree and cannot form a cycle. For xN_1, a voter with $x > y$ must have $z > x > y$. Thus a majority for $x > y$ forces a majority for $z > y$, ruling out the cycle $x > y > z > x$. For the reverse cycle, interchange y, z. The case xN_3 is the reversal of this argument. An odd profile has no majority ties. Its majority relation is a tournament, and a non-transitive tournament has a directed triangle: if $a > b$, $b > c$, but not $a > c$, then $c > a$. Every such triangle is excluded by its common law, proving transitivity. □

Lean correspondence.

`Condorcet.isCondorcet_iff_valueRestricted` is the exact semantic equivalence. Its two directions are `IsCondorcet.valueRestricted` and `ValueRestricted.isCondorcet`. The finite triple step is `Condorcet.Pattern.finite_characterization`; its finite formulation uses the six Boolean comparison patterns. The proof above is a human proof of the same equivalence, not a literal transcript of Lean's finite case split.

1.2 Maximum, maximal and full are three different notions

Definition 1.4 (Fullness and the simpler class)**.** A domain is maximal if no further order can be added while retaining the Condorcet property. It has maximum size if its size equals $f(n)$. Choose one never-law on each triple. Its full solution set is called a full domain. Define $\mathcal{F}_n$ to consist of nonempty full solution sets of one extreme law per triple together with any set of fixed pair comparisons. A fixed comparison may be absent or may require either orientation. It does not impose a common axis.

Lean correspondence.

`Condorcet.IsMaximal` includes both the semantic Condorcet property and inclusion-maximality. `Selection`, `HoldsLaw`, `fullDomain` and `IsFull` formalise full general domains. `ExtremeSelection`, `FixedPairs`, `extremeFullDomain` and `IsExtremeFull` describe the full extreme-law-plus-pairs solution set, which may be empty. `Condorcet.InF` adds nonemptiness and is exactly our $\mathcal{F}_n$. These two predicates must not be interchanged.

Lemma 1.5 (Completion)**.** *A maximum general domain is maximal, and a maximal general domain is full. A maximum peak-pit domain is a full solution set of extreme laws, and hence belongs to* $\mathcal{F}_n$*.*

Proof. Adding an order to a maximum domain would increase its size, proving maximality. For a maximal domain choose a common law on each triple using Theorem 1.3, and include every order satisfying the chosen laws. The enlargement is still Condorcet, so maximality makes it equal to the original domain. For a maximum peak-pit domain, make the choices extreme. Its completion remains peak-pit, so maximum size again forces equality. It is nonempty, since a singleton is peak-pit. There is no need for it to be maximal among *all* Condorcet domains. □

Lean correspondence.

`Condorcet.maximum_isMaximal`, `IsMaximal.isFull`, `ExtremeRestricted.completion`, `maximum_peakPit_isExtremeFull`, `maximum_peakPit_fullClass`. The last gives `IsExtremeFull`; nonemptiness is supplied where `InF` is used. The extremal functions are actual attained maxima by `exists_maximum_general` and `exists_maximum_peakPit`.

2 Idea 1: deletion turns a structural count into a recurrence

Definition 2.1 (Deletion outputs)**.** For a domain D on X, with $n = |X|$, let

$$M = |D|, \quad D_{-x} = \{\sigma \text{ with } x \text{ erased} : \sigma \in D\}, \quad M_x = |D_{-x}|, \quad \Sigma(D) = \sum_{x \in X} M_x.$$

Duplicates are removed in D_{-x}. Outputs for different deleted alternatives are counted separately, so an output is a pair (x, P).

Lean correspondence.

`Condorcet.deleteEmbedding`, `deleteDomain`, `deletionSum`. Lean deletes from $\mathrm{Fin}(k+1)$ and canonically relabels the remaining alternatives as $\mathrm{Fin}(k)$. This relabelling changes neither distinctness nor any count.

Example 2.2 (Deletion collisions)**.** Take $D = \{123, 213, 231, 321\}$. It satisfies $2N_3$, so it is Condorcet and peak-pit. Its outputs are

order	delete 1	delete 2	delete 3
123	23	13	12
213	23	13	21
231	23	31	21
321	32	31	21
distinct outputs	2	2	2

Thus $M = 4$, $\Sigma = 6$. Deleting 1 merges three orders into 23. The useful structural theorems limit how much merging can happen on average across all deletion outputs.

Expository scope.

The displayed labels and small numerical example are expository and are checked separately; no claim is made that this exact labelled example is an exported Lean theorem. The general definitions and all identities applied to it are formalised.

Lemma 2.3 (The deletion upper estimate)**.** *Restriction preserves both Condorcet and peak-pit domains. Consequently*

$$\Sigma(D) \le nf(n-1) \quad \textit{for Condorcet } D, \qquad \Sigma(D) \le nh(n-1) \quad \textit{for peak-pit } D.$$

Proof. Lift each order in any odd profile of restricted orders to one original order of D. All comparisons between surviving alternatives are unchanged. Majority is therefore the restriction of a transitive relation, hence transitive. An extreme triple law is also preserved by restriction. Each of the n deletion families is thus bounded by the relevant maximum at $n-1$; sum those bounds. □

Lean correspondence.

`Condorcet.IsCondorcet.restrict`, `IsPeakPit.restrict`, `deletionSum_le_general`, `deletionSum_le_peakPit`. The latter two literally use $k+1$ alternatives and a factor $k+1$; set $n = k+1$.

Why this idea helps. An upper bound on Σ is easy because deletions have one fewer alternative. The hard part is to prove a large *lower* bound on Σ in terms of M. If $\Sigma \geq c_n M$ for a maximum domain, the two estimates give $f(n) \leq (n/c_n) f(n-1)$. The general algebraic observation is exposition; the final numerical instance is an explicit Lean theorem.

For perspective, $M \leq \Sigma$ holds for every order family, even without the Condorcet property: count nM pairs (σ, x), and note that one output (x, P) has at most n insertion positions as preimages. This weak bound yields only a factorial recurrence. The structural argument must force Σ to be a positive fraction of nM.

3 Idea 2: runs are positions minus edges

Definition 3.1 (Insertion fibres, gaps and swaps)**.** For $P \in D_{-x}$, its insertion fibre $F(x, P)$ is the set of slots $0, \ldots, n-1$ in which reinserting x gives a member of D. Split the occupied slots into consecutive runs. A fibre with r runs contributes $r - 1$ to $G(D)$. Thus a gap counts a separation between runs, rather than every missing slot. Let $A(D)$ count pairs (σ, j) for which interchanging the adjacent alternatives in positions $j, j+1$ stays in D. An undirected swap edge contributes twice to A.

Lean correspondence.

`Condorcet.insertionSlots`, `Condorcet.insertionWord`, `SlotWord.runs`, `SlotWord.gaps`, `Condorcet.gapCount`, `directedEdges`, `adjacencyCount`. Some local definitions use Option(X): `none` denotes the inserted label and `some` the surviving labels. The transport back to labelled orders is proved, rather than assumed.

Theorem 3.2 (Exact fibre identity)**.** *For every finite order family, without any Condorcet hypothesis,*

$$\boxed{\Sigma(D) + G(D) + A(D) = nM.}$$

Proof. A fibre with m occupied slots and r runs has $m - r$ pairs of consecutive occupied slots: each run of size s has $s - 1$ edges. Summing occupied slots over all fibres counts every order once for every deleted label, giving nM. Summing runs gives $\Sigma + G$, since every nonempty fibre contributes one output and $r - 1$ gaps. Each consecutive-slot edge moves x past its neighbour. An actual swap of adjacent a, b appears once with a distinguished and once with b distinguished, precisely the twice-per-edge convention of A. Hence the sum of $m - r$ is A, proving the identity. □

Lean correspondence.

`Condorcet.deletion_gap_adjacency_identity` states `deletionSum` + `gapCount` + `adjacencyCount` $= (k+1)|D|$. The equality between occupied-slot edges and actual swaps is proved by `slotAdjacencyCount_eq_adjacencyCount`; the local run identity is `SlotWord.nonempty_run_identity`.

In Example 2.2, the swap graph is the path $123 \leftrightarrow 213 \leftrightarrow 231 \leftrightarrow 321$. Thus $A = 6, G = 0$, and $6 + 0 + 6 = 3 \cdot 4$. By contrast, $\{123, 231\}$ has $\Sigma = 5, G = 1, A = 0$. It is a peak-pit subset, but its 1-fibre above 23 has slots $\{0, 2\}$. Completion matters in the next lemma.

Lemma 3.3 (Interval fibres)**.** *For a full extreme-law-plus-pair family, every nonempty insertion fibre is an interval. Therefore $G = 0$, and $\Sigma = nM - A$.*

Proof. Fix the relative order P of all labels except x. Every defining constraint permits an interval of positions for x. For instance, xN_1 forces x below the higher of the other two labels; xN_3 forces it above the lower. If a different label y is never first, either the third label is already above y, making the condition automatic, or x must lie above y. Never-last is the reverse case. A fixed comparison involving x permits a prefix or suffix of slots; constraints not involving x are constant. Intersect these intervals. Fullness says that every surviving slot is present, so the fibre has at most one run. Apply Theorem 3.2. □

Lean correspondence.

`Condorcet.IsExtremeFull.insertionWord_interval`, `IsExtremeFull.gapCount_zero` and `IsExtremeFull.deletion_plus_slotAdjacency`. The premise is `IsExtremeFull`, not merely `IsPeakPit`.

4 Idea 3: assign flexibility to distinct obstructions

Why this idea helps. Deleting a freely movable alternative can merge many orders. The Condorcet restrictions force a compensating rigid adjacent pair elsewhere. The key is to make this assignment injective.

Definition 4.1 (Moves and their counts)**.** A lift of x moves it upwards, preserving the relative order of all other labels and staying inside D; a drop moves it downwards. Moves may jump several positions. An incidence (σ, x), with $\sigma \in D$, is free if both moves exist and frozen if neither exists. Their counts are fr, fz. A neighbouring ordered pair a, c in σ is locked if c has no lift and a has no drop. Let lk count these order/pair incidences.

Lean correspondence.

`Condorcet.HasLift`, `HasDrop`, `Free`, `Frozen`, `freeCount`, `frozenCount`, `Locked`, `lockedCount`. The exact exported move names and their source locations are recorded in the definition index. Here locked refers to arbitrary lifts and drops, not only the adjacent swap.

Lemma 4.2 (Two move counts)**.** *Every family satisfies*

$$2\Sigma = nM - \mathrm{fr} + \mathrm{fz}\,.$$

For a full domain one also has

$$2\Sigma = (n+1)M - A + \mathrm{lk}, \qquad 3\Sigma + \mathrm{fr} = (n+1)M + G + \mathrm{fz} + \mathrm{lk}\,. \tag{$*$}$$

Proof. Each nonempty fibre has exactly one highest occupied slot and one lowest. Thus Σ counts incidences with no lift, and also incidences with no drop. An incidence contributes $0, 1, 2$ to their sum according as it is free, has just one move, or is frozen. This gives the first equation.

For a neighbouring pair a, c, count the missing drop of a and missing lift of c. An enabled swap contributes zero, a locked pair contributes two. Every other blocked swap contributes one. To justify the last assertion, a blocked swap violates a defining triple law involving both a, c, since all other comparisons are unchanged. If the third label z lies above both, the changed triple is $zac \to zca$;

its only possible newly violated laws are aN_3 and cN_2. The first rules out every lift of c across a; the second rules out every drop of a across c. If z lies below both, the changed triple is $acz \to caz$, and the possible laws are aN_2, cN_1; these likewise rule out one of the crossing moves. Therefore a forbidden swap cannot have both crossing moves available. Fullness is what supplies a violated chosen law when the swapped order is absent.

There are also two end contributions per order: its first label has no lift and its last label no drop. Summing gives $2\Sigma = 2M + (n-1)M - A + \mathrm{lk}$. Combine this with the first equation and $\Sigma + G + A = nM$ to obtain $(*)$. □

Lean correspondence.

`Condorcet.first_move_count`, `IsFull.missing_swap`, `IsFull.second_move_count`, `IsFull.gap_slack_identity`. The last is written without subtraction in $\mathbb{N}$: $3\,\texttt{deletionSum} + \texttt{freeCount} = (k+2)|D| + \texttt{gapCount} + \texttt{frozenCount} + \texttt{lockedCount}$. The coefficient $k+2$ is $n+1$ because $n = k+1$.

Lemma 4.3 (A fixed pair cannot be consecutive in a maximal domain)**.** *If every order in a maximal Condorcet domain has $a > c$, then no order in the domain puts a, c consecutively.*

Proof. Otherwise interchange that consecutive pair to obtain τ. Triples not containing both labels are unchanged from the original order. On a triple containing a, c, z, all old orders satisfy cN_1 and aN_3, because $a > c$ is fixed. If z lies above the consecutive pair in the original order, τ has trace zca, which still satisfies cN_1. If z lies below, τ has trace caz, still satisfying aN_3. Thus the old family together with τ has a common law on every triple and is Condorcet. The new order reverses the fixed pair and was absent, contradicting maximality. □

Lean correspondence.

`Condorcet.IsMaximal.fixed_pair_not_consecutive`.

Proposition 4.4 (Free-to-locked injection)**.** *For a maximal Condorcet domain,* $\mathrm{fr} \le \mathrm{lk}$.

Proof. Let b be free in an order whose immediate neighbours are a, b, c. The order itself, a lift and a drop show the triple traces abc, bac, acb. The only possible common laws are aN_3 and cN_1. Both cannot hold: together they force $a > c$ in every order, while a lift of b puts a, c consecutively, contrary to Lemma 4.3.

If aN_3 holds, take the lowest available lift of b; otherwise take its highest available drop. In the resulting order τ, a, c are consecutive and locked. In the lift case, b lies above both: lifting c above a, or dropping a below c, makes a last on their triple, violating aN_3. The drop case is reversed. This defines the assignment.

For a fixed moved label b, its preimage is uniquely recovered by moving b into the slot between the image pair. It remains to exclude different moved labels with the same image. The following three cases give that exclusion explicitly.

Call the image pair p, q. A preimage is upper when its moved label b is above p in τ. Then pN_3 holds and qN_1 does not, so there exists $\chi_b \in D$ with trace qpb. For a lower preimage qN_1 holds and pN_3 does not. Suppose two distinct labels b_1, b_2 give the same image.

Both upper. Drop b_i in its original preimage σ_i, obtaining ρ_i. On $\{b_1, b_2, q\}$ the four traces are

order	σ_1	σ_2	ρ_1	ρ_2
trace	b_2b_1q	b_1b_2q	b_2qb_1	b_1qb_2.

Both b_1, b_2 occupy every position, so only qN_1 can hold. In χ_{b_1}, q is above p, b_1. Thus b_2 must be above q, giving trace b_2qp on $\{p, q, b_2\}$, contrary to its pN_3 law.

Both lower. Reverse every order. Maximality and the Condorcet property are preserved; lifts and drops exchange roles. The assignment also reverses: exactly one of its two possible triple laws holds, and the lowest lift becomes the highest drop. This reduces to the case just excluded.

One upper and one lower. Let b_1 be upper and b_2 lower. The traces on $\{p, q, b_1, b_2\}$ are

$$\tau : b_1pqb_2, \quad \sigma_1 : pb_1qb_2, \quad \sigma_2 : b_1pb_2q.$$

Take $\chi = \chi_{b_1}$, with trace qpb_1. The law qN_1 on $\{p, q, b_2\}$ forces b_2 above q, so χ has trace b_2qpb_1. On $\{b_1, b_2, q\}$, the orders τ, σ_2, χ have traces $b_1qb_2, b_1b_2q, b_2qb_1$. These permit just two common laws: b_1N_2 and qN_1. Let ρ be a drop of b_1 from σ_1. In ρ, the label q is above both b_1 and b_2: the drop puts b_1 below its neighbour q, while the comparison of q with b_2 stays fixed. Thus ρ rules out qN_1, leaving b_1N_2. Let λ be a lift of b_2 from σ_2. The remaining law forces the traces of ρ, λ on this triple to be qb_2b_1 and b_1b_2q. On $\{p, b_1, b_2\}$, the five orders $\tau, \sigma_1, \rho, \lambda, \chi$ therefore give

$$b_1pb_2, \quad pb_1b_2, \quad pb_2b_1, \quad b_1b_2p, \quad b_2pb_1.$$

Every alternative occupies every position, leaving no common law. This contradiction excludes the last collision and proves injectivity. □

Lean correspondence.

`CDCert.FreeLocked.FreePair.law`, `not_both`, `lift_locked`, `drop_locked`, followed by `CDCert.FreeLocked.psi_inj` and `free_le_locked`. The conclusion assumes only `IsMaximal`; fullness and the triple laws are obtained inside the dependency chain.

Theorem 4.5 (Gap inequality)**.** *For every maximal Condorcet domain,*

$$\boxed{G + (n+1)M \leq 3\Sigma.}$$

Proof. Use fr $\leq$ lk in Lemma 4.2, and discard the nonnegative frozen count. No finite enumeration is used. □

Lean correspondence.

`CDCert.Recurrences.gap_inequality` and its final wrapper `CDCert.TwoInputs.first_input`. The helper `CDCert.Bounds.FreeLockedInjection` is an interface, not an unproved premise: `CDCert.Recurrences.freeLockedInjection` discharges it using the injection just proved.

5 Idea 4: decompose by majority projection

Join two orders of D by an edge when they differ by one adjacent swap. Let its connected components be K. Connectivity here always means paths in this adjacent-swap graph.

Lean correspondence.

`Condorcet.AdjacentSwap`, `Connected`, `component` and `components`. These are not definitions using an arbitrary graph on the domain.

Lemma 5.1 (Majority closure)**.** *A full domain contains the majority order of every three of its orders.*

Proof. The majority relation is a linear order by the Condorcet property. It preserves each chosen triple law. A never-middle law equates two comparison bits, so majority preserves their equality. For a never-first law, if majority put the distinguished label above both others, two sets of at least two of the three voters would support the two comparisons. Their intersection contains a voter putting that label first, a contradiction. Reverse the argument for never-last. Fullness then puts the majority order in the family. Fixed pair comparisons, when present, are preserved too. □

Lean correspondence.

`Condorcet.medianOrder` constructs an actual strict order from semantic majority; `IsFull.median_mem` proves membership. The Boolean-coordinate statement is `IsFull.encoded_medianClosed`.

Lemma 5.2 (A component is determined by its fixed comparisons)**.** *Let D be full and K one component. Let U be the pair comparisons having the same value throughout K. Then*

$$K = \{\tau \in D : \tau \text{ agrees with } U\}.$$

Proof. Only the reverse inclusion needs proof. Suppose $\tau \in D$ agrees with U, and choose $\sigma \in K$ minimizing the number of pair comparisons on which it disagrees with τ. If they differ, some differing comparison is not fixed on K, so choose $\rho \in K$ agreeing with τ on that comparison. Project a path in K from σ to ρ by

$$v \longmapsto \mathrm{maj}(\sigma, \tau, v).$$

Every projected order lies in D by Lemma 5.1. Consecutive projections differ in at most the one comparison changed by the original edge. Two linear orders differing in one comparison are adjacent by a swap: if another label lay between the exchanged labels, transitivity would force some further comparison to change. Thus the projected path stays in K and starts at σ. Where σ, τ agree, projection keeps that value. Where they disagree, projection follows v. Its final point corrects the selected disagreement and introduces no disagreement on which σ was already correct. It is therefore closer to τ, a contradiction. □

Lean correspondence.

`Condorcet.IsFull.component_from_fixed` and `component_membership`. The formal proof encodes one Boolean coordinate per unordered pair; `encode_injective` proves that this representation loses no order information.

Theorem 5.3 (Component reduction and count)**.** *If D is full, every adjacent-swap component K lies in $\mathcal{F}_n$, and*

$$\boxed{\sum_K |K| = M, \qquad \sum_K \Sigma(K) = \Sigma(D) + G(D).}$$

Proof. A chosen never-middle law xN_2 equates the two comparisons of x with the other labels of that triple. A swap changes at most one of those bits, so their common value cannot change along a path. Throughout a component, x is consequently above both labels or below both. Replace this law by xN_3 in the first case and xN_1 in the second, and impose all the comparisons U fixed on the component. Retain the original extreme laws. Every order in K satisfies the resulting constraints.

Conversely an order satisfying them satisfies the old laws, since the fixed comparisons recover each replaced never-middle law. It lies in D and agrees with U, hence lies in K by Lemma 5.2. This proves $K \in \mathcal{F}_n$, including the required fullness and nonemptiness.

Components partition D, giving the first identity. In one original insertion fibre, a run lies in a single component by its consecutive swap edges. A component cannot contain two separated original runs: its own fibres are intervals by Lemma 3.3. Thus every run belongs to exactly one component and contributes one deletion output there. Counting all runs gives $\sum_K \Sigma(K) = \Sigma + G$. □

Lean correspondence.

`Condorcet.IsFull.component_inF` proves the single-component statement; `IsFull.components_inF` applies it to members of `components`. The identities are `sum_components_card` and `IsFull.sum_components_deletionSum`. Their premises are fullness, not maximality; maximality is needed later for the gap inequality.

Example 5.4 (What the extra count means)**.** The full domain $\{123, 132, 231, 321\}$, defined by $1N_2$, has components $\{123, 132\}$ and $\{231, 321\}$. Its deletion sum is 6 and its gap count 2. Each component has deletion sum 4, so the component sums add to $8 = 6 + 2$. Splitting the family counts the previously separated fibre runs separately.

6 The local inequality needed by Ideas 5 and 6

The next lemma is the one finite input to the strongest argument. We first give its mathematical proof from three exact finite facts, then justify how those facts are proved exhaustively in Section 11. This keeps the structure visible without leaving a finite assertion unaccounted for.

6.1 Endpoint slices and their closure

For a family H on k alternatives, a bottom slice fixes the last label x and then deletes it; a top slice fixes and deletes the first label. The slice retains only orders having that specified endpoint, so it is not the same as ordinary deletion from all orders. At either end the slice sizes sum to $|H|$.

Every nonempty endpoint slice of a member of $\mathcal{F}_k$ lies in $\mathcal{F}_{k-1}$. Internal constraints remain unchanged. A triple with the deleted endpoint and two surviving labels becomes a fixed pair comparison, a tautology or a contradiction. Existing comparisons involving the endpoint likewise become constant. Contradictions give an empty slice. These observations also apply after fixing an ordered prefix and suffix, one endpoint deletion at a time.

Lean correspondence.

`CDCert.K6.botP`, `topP`, `slice_bot`, `slice_top`. The list-level `IsDom` represents full extreme-law-plus-pair solution sets, with no requirement on values at lists that are not permutations. `DomE` adds families empty on the permutations. Empty slices are allowed in all sums.

For a seven-alternative family J, let $a_j(J)$ count orders with an enabled swap at positions $j, j+1$, $1 \le j \le 6$, and set

$$C(J) = \frac{a_1 + 2a_2 + 2a_3 + 2a_4 + 2a_5 + a_6}{10}.$$

If we sum $A(H)$ over all top and bottom six-slices, the first and last swaps occur once, and the four others twice. Thus

$$\sum_{\text{top and bottom six-slices } H} A(H) = 10C(J). \tag{6.1}$$

Lean correspondence.

`CDCert.K6.adjAt` uses swap indices $0, \dots, k-2$. `adj` counts A, and `cost10` is exactly $10C$. `adj6_split`, `cost10_split` prove the slice-count identities. The semantic-to-list preservation of A is supplied by the bridge in Section 11.6.

Example 6.1 (What the weighted count means)**.** On seven alternatives, allow either order inside each pair $\{1,2\}, \{3,4\}, \{5,6\}$, keep these three blocks in that order, and put 7 last. There are $2^3 = 8$ orders. The enabled swaps are exactly at positions 1, 3 and 5, in every order, so

$$(a_1, a_2, a_3, a_4, a_5, a_6) = (8, 0, 8, 0, 8, 0), \qquad C = \frac{8 + 2 \cdot 8 + 2 \cdot 8}{10} = 4.$$

This is a full extreme-law-plus-pair family: all cross-block comparisons are fixed, and every triple has a compatible extreme law. Here $A = 24$, whereas $C = 4$; they measure different things. The weights in C are chosen so that sliding windows later count each interior swap with total weight one.

Theorem 6.2 (Finite seven-window estimate)**.** *There is a specified rational function q on six-alternative extreme-law-plus-pair families, extended by zero to the empty family, such that every $J \in \mathcal{F}_7$, and also the empty family, satisfies*

$$\boxed{C(J) \le \frac{5}{8}|J| + \sum_{\text{bottom } H} q(H) - \sum_{\text{top } H} q(H).}$$

Lean correspondence.

`CDCert.K6.finite_lemma` in `CDCert/K6/SevenFinal.lean`. The function is fixed by `qD`; it is not chosen separately for each J. The theorem is integer-scaled, as detailed below. It applies to `DomE 7 J`, including every fixed pair choice and the empty family.

6.2 The weights, corrections and exact facts

It helps to view the next definitions as an accounting scheme. The quantity s is the slack in a preliminary inequality. Splitting it into θ_b and θ_t allocates that slack to the two ends of a slice. The correction p transfers value between those two accounts without changing their sum. The quantity q is what remains after the internal contributions cancel. The calculations below establish these descriptions exactly.

The certificate specifies rational weights ϕ on five-label isomorphism types and nine exceptional correction values. These are explicit finite data, included in the companion Lean source; they are not additional existence assumptions. Put

$$b(H) = \sum_{\text{bottom five-slices } P} |P|\phi(P), \qquad t(H) = \sum_{\text{top five-slices } P} |P|\phi(P),$$

$$s(H) = \frac{12}{19}|H| + b(H) - t(H) - \frac{1}{5}A(H), \qquad \eta = \frac{1}{152}.$$

A specified correction $p(H)$ defines

$$\theta_b(H) = s(H)/2 + p(H), \quad \theta_t(H) = s(H)/2 - p(H), \quad q(H) = (b(H) + t(H))/2 + p(H).$$

Outside a set $\mathcal{S}$ of nine six-label types, $p(H) = 2\eta(r_t(H) - r_b(H))$. Here r_b counts bottom five-slices of one specified five-label type and r_t counts top five-slices of a second specified type. On $\mathcal{S}$, the nine listed constants replace this formula. A further set $\mathcal{T}$ consists of five types, disjoint from $\mathcal{S}$.

The exact finite facts are:

F1. For the eight-order type $R_8 \in \mathcal{T}$, $\theta_b = \theta_t = 0$.

F2. Outside $\mathcal{T} \cup \mathcal{S}$,

$$\theta_b(H) \geq \frac{\eta}{2}|H| + 4\eta r_t(H), \qquad \theta_t(H) \geq \frac{\eta}{2}|H| + 4\eta r_b(H).$$

F3. If some six-slice of J has type in $(\mathcal{T} \setminus \{R_8\}) \cup \mathcal{S}$, then

$$\Lambda(J) := \sum_{\text{bottom } H} \theta_b(H) + \sum_{\text{top } H} \theta_t(H) \geq \eta|J|.$$

The type $R_8 \in \mathcal{T}$ has eight orders on six alternatives:

$$R_8 = \{524130, 524310, 534120, 534210, 542130, 542310, 543120, 543210\}.$$

The specified five-types in r_b, r_t are respectively obtained by deleting its first and its last label. Its orders all start in 5 and end in 0, and direct calculation gives $r_b(R_8) = r_t(R_8) = 0$. The subscript denotes its eight orders, not eight alternatives.

Lean correspondence.

Definitions: `CDCert.K6.potW`, `bD`, `tD`, `sD`, `pD`, `thB`, `thT`, `qD`, `lamD`, `rb`, `rt`, `IsS`, `excRep6`. The two five-types have formal indices 866 and 41. The combined six-family check is `CDCert.K6.six_sem`; the regular receiver inequalities are `receiver_slack`, `thB_bound` and `thT_bound`. F3 is proved by `seven_bot`, `seven_top` after `relabel_slice`, using all 26 theorems `seven_fam_0` through `seven_fam_25`. `r8_th` proves F1; `r8_facts` and `r8_slices` provide the remaining stated R8 properties.

6.3 Idea 10: pay for the remaining exceptional slices

Completion of the proof of Theorem 6.2. F3 already proves $\Lambda(J) \geq \eta|J|$ unless every six-slice is regular, empty, or of type R_8. For a regular slice, F2 supplies a baseline $\eta|H|/2$ and extra contributions. An R_8 slice has value zero by F1 and is missing baseline 4η.

Suppose a bottom R_8 slice is obtained by fixing last label z. All its eight orders have the same first label w. They therefore also belong to one top receiver slice at w. In that receiver, fixing last label z gives the five-type counted by r_b, so F2 supplies an extra 4η in its θ_t value. The receiver cannot be R_8, whose r_b is zero, and cannot be empty. It is therefore regular. Different bottom R_8 slices have different last labels, hence charge different receiver/last-label pairs. They do not reuse a contribution. Reverse top and bottom for top R_8 slices; those charges use the separate θ_b contributions.

All missing baselines are thus paid. At each end the slice sizes sum to $|J|$, giving total baseline $\eta|J|$. Therefore $\Lambda(J) \geq \eta|J|$ in every case.

Now expand Λ. The terms on five-slices obtained by deleting one first and one last label cancel, because the two deletions commute, with the corresponding labels canonically relabelled. Equation (6.1) gives the remaining swap contribution. The resulting exact identity is

$$\Lambda(J) = \frac{12}{19}|J| - C(J) + \sum_{\text{bottom } H} q(H) - \sum_{\text{top } H} q(H).$$

Finally $12/19 - \eta = (96-1)/152 = 5/8$, proving the claimed inequality. The empty-family case has every term zero. □

Lean correspondence.

`CDCert.K6.lam_receiver` is the receiver argument; `seven_finite` combines it with F3. The cancellation identity is `lamD_eq`. Thus neither the exceptional R8 family nor coverage of all seven-families is left as a premise of `finite_lemma`. Section 11 supplies the finite proofs of F1–F3, completing the dependency chain.

6.4 The exact integer normalisation

Let $d = 8\,846\,117\,280$. Formal variables are normalised as follows:

Formal quantity	Mathematical quantity
`cD5`	$d/5 = 1\,769\,223\,456$
`cK`, `cE`	$12d/19 = 5\,587\,021\,440$, $d/152 = 58\,198\,140$
`cost10`	$10C$
`sD`, `bD`, `tD`	ds, db, dt
`pD`, `thB`, `thT`, `qD`	$2dp, 2d\theta_b, 2d\theta_t, 2dq$
`lamD`	$2d\Lambda$

Multiplying Theorem 6.2 by $2d$ gives literally

$$(d/5)\,\texttt{cost10}(J) \le 11\,057\,646\,600\,|J| + \sum_{\text{bottom } H} \texttt{qD}(H) - \sum_{\text{top } H} \texttt{qD}(H).$$

This is the statement of `CDCert.K6.finite_lemma`. Counts are cast into integers before subtraction. Consequently there is no truncated subtraction of a possibly negative potential in this step.

7 Ideas 5 and 6: local windows and telescoping corrections

Why this idea helps. A seven-position window can have an unusually large swap count. Its end corrections account for the excess. When the window slides one position, its new overlap is the old overlap, so the accounting corrections cancel inside the order.

7.1 See one overlap before summing all windows

Picture eight consecutive positions as

$$x \mid a\ b\ c\ d\ e\ f \mid y.$$

The first seven-position window is $xabcdef$; the next is $abcdefy$. Fix the full exterior order in each application. In the first window, fixing and deleting the first label x leaves the central six positions. In the second, fixing and deleting the last label y leaves exactly the same six-position family, with the same ordered exterior. Its correction occurs with a minus sign in the first inequality and a plus sign in the second. This is why it cancels. The matching remembers the fixed exterior; it does not identify unrelated families merely because they look alike.

The weights can also be seen on eight positions. In the following table all entries have denominator 10:

Swap position	1	2	3	4	5	6	7
First window	1	2	2	2	2	1	0
Second window	0	1	2	2	2	2	1
Total	1	3	4	4	4	3	1

There are seven possible swap positions, and the total weight is $20/10 = 2$. The missing weight is $7 - 2 = 5$. For a longer order the middle positions eventually receive weight one; the same total deficit of five is confined to the outside ends. The proof below works uniformly even when the order is short.

7.2 The complete count

Proposition 7.1 (A global estimate with two outside ends)**.** *There is a fixed constant B, independent of n and K, such that for every $K \in \mathcal{F}_n$ with $n \geq 7$,*

$$A(K) \leq \frac{5}{8} n|K| + B|K|, \qquad \Sigma(K) \geq (3n/8 - B)|K|.$$

Proof. At each window position fix the full ordered prefix and suffix outside the seven consecutive positions. The remaining labels and their allowed orders form a conditional family $J \in \mathcal{F}_7$, unless empty, by endpoint-slice closure. These conditional families partition K, so their sizes sum to $M = |K|$. Different exterior contexts are retained with their multiplicities even if their local families are isomorphic. Apply Theorem 6.2 to each context and sum.

The top six-slices of the window in positions $i, \ldots, i+6$ match the bottom six-slices of the next window. Both retain positions $i+1, \ldots, i+6$ and fix the same ordered exterior. Consequently all internal q-terms cancel. Formally this is the elementary sum $\sum_{i=0}^{m-1}(Q_i - Q_{i+1}) = Q_0 - Q_m$, with the sign convention bottom minus top. The two remaining six-position layers each have total size M. There are finitely many six-label families and $q(\varnothing) = 0$, so choose Q with $|q(H)| \leq Q|H|$. The boundary contribution has absolute value at most $2QM$. Thus

$$\sum_{\text{all window contexts } J} C(J) \leq \frac{5}{8}(n-6)M + 2QM.$$

Every interior swap position has total weight $(1+2+2+2+2+1)/10 = 1$ across windows. At all positions the total weight is between zero and one. There are $n-1$ swap positions, but the sum of the weights over the $n-6$ windows is $n-6$. The total missing weight is therefore exactly 5 per order. Since a swap indicator is at most one,

$$A(K) \leq \sum_J C(J) + 5M \leq \frac{5}{8} nM + \left(\frac{5}{4} + 2Q\right) M.$$

Take $B = 5/4 + 2Q$. Interval fibres give $\Sigma = nM - A$, proving the second assertion. □

Lean correspondence.

`CDCert.K6.winS_card` and `winS_adjAt` identify the window counts; `winS_bot`, `winS_top` identify the matching six-position contexts. `window_ineq` applies the finite estimate; `isum_telescope` cancels the internal terms. The final scaled bound is `CDCert.K6.window_bound`:

$$8dA \le 5dnM + (10d + 2^{66})M.$$

`qD_bound` supplies $|\mathtt{qD}(H)| \le 2^{63}|H|$, so $Q = 2^{62}/d$ and $B = 5/4 + 2^{63}/d$ recover exactly this coefficient. The enormous boundary constant is harmless; the next idea removes it.

8 Idea 7: repeated copies remove the bounded error

Definition 8.1 (Ordered concatenation)**.** Take disjoint label blocks. Put every label of an earlier block above every label of a later one, and choose an order from the given family inside each block independently. For integers $t \ge 1$, write $K^{[t]}$ for t copies of one family K.

Lemma 8.2 (Exact product counts)**.** *If $n, t \ge 1$, $K \in \mathcal{F}_n$, $M = |K| > 0$, then $K^{[t]} \in \mathcal{F}_{tn}$, and*

$$|K^{[t]}| = M^t, \quad A(K^{[t]}) = tA(K)M^{t-1}, \quad \Sigma(K^{[t]}) = t\Sigma(K)M^{t-1}.$$

Proof. Retain all internal defining constraints and fix every comparison between blocks. A triple meeting more than one block has a label fixed first or last, so a compatible extreme law can be chosen. The constraints describe exactly the concatenated orders, proving membership in $\mathcal{F}$. Independent choices give the first formula. An enabled internal swap can occur in any of the t blocks while the other $t-1$ blocks have M^{t-1} choices. A swap across a join violates a fixed cross-block comparison and is never enabled. This proves the second formula. Deleting label x in a block similarly gives $M_x M^{t-1}$ outputs. Their blocks are recoverable from their disjoint labels and forced ordering, so there is no extra identification or correction at a join. Sum over all blocks and labels. □

Lean correspondence.

`CDCert.K6.concat`, `isDom_concat`, `card_concat`, `adj_concat` formalise the construction and first two counts at list level. `dbl` and `dbl_props` iterate doubling, using $t = 2^k$. The displayed deletion product is also a written counting proof; the formal amplification uses the adjacency product instead, then the fibre identity. We do not label the deletion-product formula as a separately exported Lean theorem.

For instance $K = \{12, 21\}$ gives $K^{[2]} = \{1234, 1243, 2134, 2143\}$. Each deletion leaves two orders, so $\Sigma = 8 = 2 \cdot 2 \cdot 2$; the swap at the join is absent.

8.1 A slightly larger construction that can be inspected

Use the four-order domain $D = \{123, 213, 231, 321\}$ from Section 2, and a relabelled copy $D' = \{456, 546, 564, 654\}$. Joining them means choosing a prefix from D and a suffix from D'. There are 16 orders, including $123456, 123546, 213564, 321654$. Every deletion has eight distinct outputs: two choices inside its affected block, and four in the other. Thus $\Sigma = 6 \cdot 8 = 48$. The two internal path graphs contribute 24 undirected edges, hence $A = 48$. There is no swap across the join. All fibres are intervals, so $G = 0$ and $48 + 0 + 48 = 6 \cdot 16$.

The same recipe gives the following genuine domains. Write $n = 3t + r$, $r \in \{0, 1, 2\}$; join t copies of D, then append one fixed label if $r = 1$, or a freely ordered pair if $r = 2$. Each row counts this constructed family, without claiming it is largest.

n	Three-label blocks	M	Σ	G	A
5	1	8	20	0	20
6	2	16	48	0	48
7	2	16	64	0	48
8	2	32	128	0	128
9	3	64	288	0	288
10	3	64	352	0	288

This table makes the absence of a new join correction visible in actual counts. The construction and its counts are checked by the separate example script; the general product and interval identities used above are the formalised ones already identified.

8.2 Remove the outside error

Theorem 8.3 (Exact extreme-law density)**.** *For $n \geq 1$ and $K \in \mathcal{F}_n$,*

$$\boxed{8A(K) \leq 5n|K|, \qquad 3n|K| \leq 8\Sigma(K).}$$

Proof. Apply Proposition 7.1 once to the whole joined family, for t large enough that $tn \geq 7$:

$$tA(K)M^{t-1} \leq \frac{5}{8}tnM^t + BM^t.$$

Divide by $tM^{t-1} > 0$, obtaining $A(K) \leq (5n/8 + B/t)M$. Taking arbitrarily large t, or just powers of two, gives the first inequality. Interval fibres and $\Sigma + A = nM$ give the second. The original n can be small, since only the repeated family's length must reach seven. □

Lean correspondence.

`CDCert.K6.adj_le` proves the first inequality at list level. Lean avoids a real limit: if the integer bound failed by at least one, choose 2^k larger than the fixed error can permit, use `dbl_props`, and obtain a contradiction. After transferring lists back to actual orders and using gap zero, `CDCert.Bridge.InF.density` states the second inequality on $\mathrm{Fin}(k+1)$. Both are among the 20 audited results.

9 The transfer and all seven final conclusions

Theorem 9.1 (The second input)**.** *For every full domain D on $n \geq 1$ alternatives,*

$$\boxed{3nM \leq 8(\Sigma + G).}$$

Proof. Apply Theorem 8.3 to each component and sum. By Theorem 5.3, the cardinalities sum to M and the deletion sums to $\Sigma + G$. The empty-domain case has both sides zero. □

Lean correspondence.

`CDCert.TwoInputs.second_input`, literally on $\mathrm{Fin}(k+1)$, with fullness as its only domain premise.

Theorem 9.2 (Transfer to general domains)**.** *For every maximal Condorcet domain on $n \geq 1$ alternatives,*

$$\boxed{(11n+8)M \leq 32\Sigma.}$$

Proof. Maximality supplies fullness. The second input and the gap inequality give $3nM/8 \leq \Sigma + G \leq 4\Sigma - (n+1)M$. Multiply by eight and rearrange: $(3n+8n+8)M \leq 32\Sigma$. □

Lean correspondence.

`CDCert.TwoInputs.transfer` and `CDCert.Recurrences.deletion_lower_bound`. There is no assumed injection or density theorem in these statements: both have been proved in the dependency chain. The displayed rational algebra is performed over natural counts after clearing denominators.

Theorem 9.3 (The seven final entry-point conclusions)**.** *Let f, h have the semantic definitions in Section 1. The first four assertions hold for every integer $n \geq 3$.*

E1. $(11n+8)f(n) \leq 32nf(n-1)$.

E2. $f(n) \leq 4(32/11)^{n-3}$.

E3. $3h(n) \leq 8h(n-1)$.

E4. $h(n) \leq 4(8/3)^{n-3}$.

E5. There is a real number L_f such that $f(n)^{1/n} \to L_f$ and $L_f \leq 32/11$.

E6. There is a real number L_h such that $h(n)^{1/n} \to L_h$ and $L_h \leq 8/3$.

E7. $f(n) = O\big(n^{-8/11}(32/11)^n\big)$ *as* $n \to \infty$.

Lean correspondence.

These are exactly, in the same order, `CDCert.Main.general_recurrence`, `general_geometric`, `peak_recurrence`, `peak_geometric`, `general_limit`, `peak_limit`, `general_polynomial_correction`. The limit assertions quantify existentially over a real limit; they assert neither an exact value nor convergence of successive ratios. The final three statements have no extra mathematical hypotheses.

9.1 Recurrences and the base value

Proof of E1–E4. Choose a domain of size $f(n)$. It is maximal, so Theorem 9.2 and Lemma 2.3 give E1. For a maximum peak-pit domain, completion puts it in $\mathcal{F}_n$. The density theorem and the peak-pit deletion estimate give E3.

On three alternatives a common never-law excludes two of the six orders, so every Condorcet domain has at most four orders. The peak-pit example in Section 2 attains four. Hence $f(3) = h(3) = 4$. Starting there, the general recurrence factor satisfies $32n/(11n+8) \leq 32/11$, so induction gives E2. Induction using E3 gives E4. The shift by three and multiplier four come from this base value. The shifted formulas are not claimed at $n = 1, 2$, where both functions take the direct values 1 and 2. □

Lean correspondence.

`Condorcet.f_three`, `h_three`; `CDCert.Recurrences.general_recurrence'` and `CDCert.Recurrences.peakPit_recurrence` actually give the recurrences for $n \geq 1$. The final `Main` wrappers use the paper's $n \geq 3$ interface. `f_geometric`, `CDCert.Recurrences.h_geometric` first prove the integer-scaled power bounds; `CDCert.Asymptotics.f_le_real` and `h_le_real` give the real-valued forms.

9.2 Root-growth limits: a separate use of concatenation

Proof of E5 and E6. A singleton domain shows $f(n), h(n) \geq 1$. Ordered concatenation of any two Condorcet domains is again Condorcet: majority restricts to transitive majorities inside the blocks and always puts the first block above the second. For peak-pit domains, a mixed triple has a label fixed first or last, so an extreme law exists. Cardinalities multiply. Thus, for $g = f$ or h,

$$g(m+n) \geq g(m)g(n).$$

Put $a_n = \log g(n) \geq 0$; it is superadditive. For a fixed $k \geq 1$, write $n = qk + r$, $0 \leq r < k$. Repeated superadditivity gives $a_n \geq qa_k + a_r \geq qa_k$, hence $\liminf a_n/n \geq a_k/k$. Taking the supremum over k shows that the liminf is at least the supremum of all a_k/k, whereas the limsup cannot exceed that supremum. The exponential upper bound makes it finite. Therefore a_n/n has a limit. The bounds E2 or E4 imply that this limit is at most $\log(32/11)$ or $\log(8/3)$. Exponentiating proves E5 and E6. □

Lean correspondence.

`CDCert.Supermult.f_supermul`, `h_supermul`, `one_le_f`, `one_le_h`; then `CDCert.Asymptotics.root_limit`, `f_root_limit`, `h_root_limit`. The generic root-limit proof uses Mathlib's subadditive machinery on the negative logarithm. This limit argument is not needed to obtain E1–E4.

9.3 The polynomial improvement, with a direct proof

Proof of E7. Set $r = 32/11$, $a = 8/11$. The recurrence is

$$f(n+1) \leq r\frac{n+1}{n+1+a}f(n).$$

For $0 < a < 1$ and $x \geq 0$, the elementary concavity inequality $(1+x)^a \leq 1 + ax$ follows by differentiating $1 + ax - (1+x)^a$: its derivative is nonnegative and its value at zero is zero. With $x = 1/(n+1)$, it gives

$$(n+2)^a(n+1) \leq (n+1)^a(n+1+a).$$

Consequently $f(n)(n+1)^a/r^n$ is non-increasing for $n \geq 3$. Its initial value is $C = 4 \cdot 4^a/r^3$, so $f(n) \leq C(n+1)^{-a}r^n \leq Cn^{-a}r^n$ for all $n \geq 3$. This is precisely the asserted big-O estimate. □

Lean correspondence.

`CDCert.Asymptotics.bernoulli_step`, `cW`, `f_weighted`, `f_isBigO`; the last is restated by `CDCert.Main.general_polynomial_correction`. The real-power exponent is exactly $-8/11$, not a natural-number subtraction. The finitely many initial values are irrelevant to `Asymptotics.IsBigO` at `atTop`.

10 What successive ideas achieve at concrete sizes

The following tables use exactly the same values of n throughout. They count *orders*, not voters. An upper-bound entry need not be attained by a domain. Our small constructed examples illustrate the counts; these tables limit *every* domain.

10.1 First compare the arguments without classification data

Use only $f(3) = 4$ as the starting value. There are three stages:

1. **Deletion alone.** The weak count $M \le \Sigma$ gives $f(n) \le nf(n-1)$, hence $f(n) \le 4n!/6$.
2. **The gap argument.** Discarding $G \ge 0$ from $G + (n+1)M \le 3\Sigma$ gives $f(n) \le 3nf(n-1)/(n+1)$.
3. **The full argument.** The local estimate and components also give $f(n) \le 32nf(n-1)/(11n+8)$. At each step use the stronger of this and the gap-only step.

Round down after *each* recurrence step, since domain sizes are integers. Precisely, set $u_3 = v_3 = 4$ and, for $n \ge 4$,

$$u_n = \left\lfloor \frac{3n}{n+1} u_{n-1} \right\rfloor, \qquad v_n = \min\left\{ \left\lfloor \frac{3n}{n+1} v_{n-1} \right\rfloor, \left\lfloor \frac{32n}{11n+8} v_{n-1} \right\rfloor \right\}.$$

Induction proves $f(n) \le u_n$ and $f(n) \le v_n$. These retain the finite correction in the recurrence, so are sharper than simply substituting n into $4(32/11)^{n-3}$.

n	Deletion alone	Gap argument	Full argument
5	80	22	22
6	480	56	56
7	3 360	147	147
8	26 880	392	392
9	241 920	1 058	1 055
10	2 419 200	2 885	2 861
20	1.622×10^{18}	89 223 302	77 257 942
30	1.769×10^{32}	3.570×10^{12}	2.523×10^{12}
100	6.222×10^{157}	2.743×10^{45}	3.097×10^{44}

Long entries are rounded *upwards* to four significant digits, so the displayed numbers are still valid upper bounds. The complete integers are provided in `data/upper_bounds_exact.csv`.

The two recurrence factors agree at $n = 8$. The gap-only factor is smaller for $n < 8$; the 32/11 factor is smaller for $n > 8$. This explains why the last two columns agree at small n. The asymptotic improvement becomes more visible as n increases.

10.2 What the separately known small maxima improve

The published classifications give

$$f(5) = 20, \qquad f(6) = 45, \qquad f(7) = 100, \qquad f(8) = 224.$$

The first three values come from the classification through seven alternatives [7]; the eight-alternative value is Theorem 4.1 of [8]. Use these as additional upper bounds at their respective steps, then continue the same two recurrences. This changes the starting information, while the proof steps stay the same.

n	Gap argument, with classifications	Full argument, with classifications
5	20	20
6	45	45
7	100	100
8	224	224
9	604	602
10	1 647	1 632
20	50 935 157	44 070 458
30	2.038×10^{12}	1.439×10^{12}
100	1.566×10^{45}	1.767×10^{44}

The rows through eight now contain exact classified maxima. The later rows remain upper estimates, and are not claims of exact maxima. In particular, knowing $f(8) = 224$ helps the finite bounds: for $n \geq 8$, even the simple geometric consequence becomes

$$f(n) \leq 224(32/11)^{n-8}.$$

It improves the leading constant, rather than the exponential base. Thus the eight-alternative classification is useful additional information, while the proof of the base $32/11$ does not require it.

Expository scope.

The recurrences used here are consequences of the formal gap and transfer results and the deletion upper estimate. These tables are separate exact-arithmetic calculations, not new exported Lean theorems. `scripts/build_numerical_examples.py` regenerates their exact entries, checks the small constructed domains and rounds long entries upwards. The published classifications in the second table are external inputs; they have not been added to the assumptions or dependency chain of the Lean proof.

11 Ideas 8–12: how the finite input becomes a proof

11.1 Idea 8: search for weights, not just large examples

A useful search describes a small family by a vector of counts: its size, available swaps, endpoint types and selected smaller slices. The desired inequality is linear in unknown weights on those counts. A linear program can seek weights making all current constraints valid. Tight or failing families suggest which additional statistics or corrections may be useful. This explains the purpose of the five-type potential and the exceptional correction.

The discovery record reports floating-point linear programming and an integer correction. It does not document every possible intermediate heuristic. Trying candidates, keeping useful choices, and examining counterexamples is a legitimate discovery strategy, but its selection history is not a theorem premise. The final proof freezes explicit rational data and proves every required assertion

about those data. The coefficient 5/8, rather than a small-domain maximum size, is what feeds the asymptotic bound.

Expository scope.

The historical search method and this explanatory description of a possible iterative workflow are not Lean theorems. Lean proves properties of the fixed data that the search eventually supplied.

11.2 Idea 9: exhaustive constraint generation

An extreme-law-plus-pair family is specified by one of six extreme laws on each triple and one of three choices on each pair: absent, first orientation, second orientation. Regard every choice as the subset of permutations it permits. The full solution set is the intersection of the selected subsets. Therefore exploring all such choices covers every family under consideration, including non-maximal ones. Conflicting choices may give the empty family. Maximal-domain classifications alone would not prove this coverage.

Here is the basic checker in mathematical pseudocode:

$$\begin{aligned} \mathrm{Check}(S, []) &= \mathrm{Leaf}(S), \\ \mathrm{Check}(S, [\mathcal{O}_1, \ldots, \mathcal{O}_m]) &= \bigwedge_{O \in \mathcal{O}_1} \mathrm{Check}(S \cap O, [\mathcal{O}_2, \ldots, \mathcal{O}_m]). \end{aligned}$$

Empty intersections can be accepted immediately when the target inequality is zero on the empty family. Correctness is induction on the remaining groups: each actual choice selects one branch, and its final solution set is the intersection at that leaf. Thus a true checker result proves the property for every choice. Deduplicating identical states or relabelling to a proved representative is safe; sampling only promising branches would not be.

Lean correspondence.

`CDCert.IsChoice`, `domainOf`, `allows`, `CDCert.ListOrder.groupsEP` describe the constraints. The semantic bridge `CDCert.Bridge.isChoice_groupsEP_iff` shows exactly which law/pair choices the generator represents. The soundness files include `CDCert/K6/Sound.lean`, `DfsPSem.lean` and `FamSem.lean`; the specialised depth-first search checks are connected to the abstract choices by proved lemmas, not by accepting the output of a native program.

11.3 Coverage on five, six and seven alternatives

At five alternatives the fixed table covers 1160 nonempty types. For any defining system on five labels, restrict its constraints to four labels, classify that full four-label solution set, then add back the constraints involving the new label. Restricting *constraints* is essential: projecting the permitted orders can lose the full constraint description. The complete proof is `CDCert.K6.five_cover`; it gives a relabelling to one of the 1160 representatives. The initial four-level classification is also proved in the project.

For a six-family, restrict its defining constraints to five labels. That full five-family appears in the table, even if it is larger than the projection of the six-family. Reintroduce every possible new extreme triple law and every new pair choice. One branch is the original six-family. Kernel-evaluated checks certify either the regular inequality or membership in the complete exceptional list. The theorem `CDCert.K6.six_sem` is the universal result; its semantics identify the regular residual with

$$ds(H) - \eta d\big(|H| + 4r_b(H) + 4r_t(H)\big).$$

Thus a nonnegative residual gives F2 after substituting the correction p, while the stored exceptional types provide F1 and the remaining seven-level cases. All table fields are proved to be the sizes, adjacencies, slice values and type indicators they claim to encode.

For seven alternatives, F3 requires every family with an exceptional six-slice other than R_8. Relabel that slice so its removed endpoint is label 6 and its remaining labels match the exact stored six-mask. Enumerate every defining constraint choice compatible with that exact endpoint slice. This is done for all thirteen relevant six-types at both top and bottom, giving 26 oriented seed families. The theorems `seven_fam_0`, ..., `seven_fam_25` prove $\Lambda \geq \eta|J|$ under the corresponding exact-slice hypothesis. `relabel_slice`, `seven_bot` and `seven_top` transport these results to every relevant slice at any label. Relabelling preserves cardinality and Λ, so it loses no case.

The formal finite enumeration has 16888 final states. These are counts of states in this proof, not a value of $f(7)$. The independent native implementation uses different reductions and has different counts. Agreement of those literal counts is not needed: completeness and the same universal inequality are what matter.

11.4 Idea 10: exceptions are mathematical information

The regular estimate intentionally permits a small specified list of failures. Thirteen types are covered by the oriented seven-family checks. The remaining R_8 case is proved by a hand argument: its common first and last labels force a receiving slice with a specific extra five-slice. Different deficits have distinct receiver/label pairs. Section 6 proves this charging argument in full. The computer-supported proof is therefore a combination of exhaustive finite checks and a general structural explanation of the one remaining exception, not merely a list of successful inequalities.

11.5 Idea 11: exact certificates and a small trusted kernel

All rational coefficients have the common denominator d. The finite checks use integer residuals and prove nonnegativity exactly. In Lean, the large generated checks use kernel reduction, including `decide +kernel`; they do not use `native_decide`, a floating-point tolerance or an axiom saying that an external checker succeeded. Generic soundness theorems turn those finite evaluations into the universal mathematical statements. The full source project includes every generated branch and literal table used in that chain. It requires no unprovided generator to understand the statements or to recompile their proofs.

11.6 Idea 12: prove that the representation preserves the mathematics

The semantic definitions use Boolean comparison relations; the finite checker uses permutations as lists. This difference must be proved harmless. Sorting a semantic order gives a list toList(r) containing every label exactly once. Reading precedence in that list recovers exactly the original comparison relation. The inverse ofList constructs a strict total order from a permutation. The two conversions are inverse, so no order is lost and no duplicate is introduced. A never-law or fixed pair holds for an order exactly when the corresponding list option accepts its list. Consequently full solution sets, their cardinalities and adjacent swaps agree on both sides. Endpoint deletion also uses the explicit canonical relabelling already described.

Lean correspondence.

`CDCert.Bridge.before_toList`, `ofList_toList`, `toList_ofList`, `toList_injective`, `lawOpt_toList`, `pairOpt_toList`, `mem_extremeFullDomain_iff`, `InF.toList_domain`, `toList_swapLetter` and `card_adj_of_choice` establish these connections. The final theorem `CDCert.Bridge.InF.density` uses those equalities, `CDCert.K6.adj_le`, the semantic fibre identity and gap zero. Its conclusion is about actual semantic orders, not merely a model that resembles them.

The native Python/C++ programs are useful independent replay tools, but their implementation correctness, timings, literal counts and search history are not propositions proved by this Lean project. The main theorem relies on the separately proved Lean enumeration and its semantic bridge.

A Complete definition and entry-point index

The two default entry modules in `lakefile.toml` are `CDCert.Main` and `CDCert.TwoInputs`. The first exports seven results and the second three. The following list includes every one of these ten declarations separately, even when a wrapper repeats a supporting result. It then lists all ten additional results in the independent axiom audit. Throughout the table, $n = k + 1$ for domain statements on $\mathrm{Fin}(k + 1)$.

The companion `audit/TUTORIAL_LEAN_MAP.json` gives the exact source file, line, declaration signature, tutorial location and normalisation for every entry. `audit/ENTRYPOINT_SIGNATURES.txt` contains the literal source signatures; the earlier compiler-printed statements and axiom lists are preserved separately. The names below are declarations, not a list inferred only from file comments.

A.1 All ten final exported declarations

Entry and Lean declaration	Exact mathematical content and hypothesis
L01 `CDCert.Main.general_recurrence`	$n \geq 3$: $(11n + 8)f(n) \leq 32nf(n - 1)$. See Theorem 9.3.
L02 `CDCert.Main.general_geometric`	$n \geq 3$: $f(n) \leq 4(32/11)^{n-3}$. See Theorem 9.3.
L03 `CDCert.Main.peak_recurrence`	$n \geq 3$: $3h(n) \leq 8h(n - 1)$. See Theorem 9.3.
L04 `CDCert.Main.peak_geometric`	$n \geq 3$: $h(n) \leq 4(8/3)^{n-3}$. See Theorem 9.3.
L05 `CDCert.Main.general_limit`	No extra premises: $\exists L_f$, $f(n)^{1/n} \to L_f \leq 32/11$. See Theorem 9.3.
L06 `CDCert.Main.peak_limit`	No extra premises: $\exists L_h$, $h(n)^{1/n} \to L_h \leq 8/3$. See Theorem 9.3.
L07 `CDCert.Main.general_polynomial_correction`	No extra premises: $f(n) = O(n^{-8/11}(32/11)^n)$. See Theorem 9.3.
L08 `CDCert.TwoInputs.first_input`	D maximal, $n = k + 1$: $G + (n + 1)M \leq 3\Sigma$. See Theorem 4.5.
L09 `CDCert.TwoInputs.second_input`	D full, $n = k + 1$: $3nM \leq 8(\Sigma + G)$. See Theorem 9.1.

Entry and Lean declaration	Exact mathematical content and hypothesis
L10 `CDCert.TwoInputs.transfer`	D maximal, $n = k + 1$: $(11n + 8)M \leq 32\Sigma$. See Theorem 9.2.

A.2 The ten additional audited supporting results

Entry and Lean declaration	Exact mathematical content and hypothesis
L11 `CDCert.Recurrences.gap_inequality`	D maximal: $G + (n + 1)M \leq 3\Sigma$. See Theorem 4.5.
L12 `CDCert.Recurrences.deletion_lower_bound`	D maximal: $(11n + 8)M \leq 32\Sigma$. See Theorem 9.2.
L13 `CDCert.Bridge.InF.density`	$K \in \mathcal{F}_n$, $n = k + 1$: $3n\lvert K\rvert \leq 8\Sigma(K)$. See Theorem 8.3.
L14 `CDCert.FreeLocked.free_le_locked`	D maximal: $\mathrm{fr}(D) \leq \mathrm{lk}(D)$. See Proposition 4.4.
L15 `CDCert.K6.finite_lemma`	`DomE`$(7, J)$: the integer-scaled seven-window inequality. See Theorem 6.2.
L16 `CDCert.K6.adj_le`	`IsDom`(n, K): $8A(K) \leq 5n\lvert K\rvert$. See Theorem 8.3.
L17 `CDCert.Supermult.f_supermul`	All $m, n \in \mathbb{N}$: $f(m)f(n) \leq f(m + n)$. See Theorem 9.3.
L18 `CDCert.Supermult.h_supermul`	All $m, n \in \mathbb{N}$: $h(m)h(n) \leq h(m + n)$. See Theorem 9.3.
L19 `CDCert.Recurrences.general_recurrence`	All $k \in \mathbb{N}$: $(11(k + 1) + 8)f(k + 1) \leq 32(k + 1)f(k)$. See Theorem 9.3.
L20 `CDCert.Bounds.peakPit_recurrence`	All $k \in \mathbb{N}$: $3h(k + 1) \leq 8h(k)$. See Theorem 9.3.

A.3 Definitions and representation conventions

The following groups identify the mathematical objects whose meanings must match. Their full source locations and signatures, together with all named helper lemmas beside the proofs, are in the machine-readable map.

Tutorial object	Lean definitions and precise convention
Orders and domains	`Condorcet.IsStrictTotal`, `Order`, `Domain`: Boolean strict total relations, finite sets of distinct orders.
Majority and Condorcet	`Condorcet.majority`, `IsCondorcet`: strict majority in every odd profile; repetitions permitted.
Never-laws	`Condorcet.Never`, `ValueRestricted`, `ExtremeRestricted`: forbidden positions 0,1,2; extremes 0 and 2.
Peak-pit and extrema	`Condorcet.IsPeakPit`, `f`, `h`: semantic domains and actual finite maxima, with no shared axis.

Tutorial object	Lean definitions and precise convention
Full and maximal	`Condorcet.IsFull`, `IsMaximal`: full solutions of one selected law per triple, versus inclusion-maximal semantic domain.
Extreme full class	`Condorcet.IsExtremeFull`, `InF`: extreme laws plus optional fixed pair comparisons; only `InF` requires nonemptiness.
Deletions and gaps	`Condorcet.deleteDomain`, `deletionSum`, `insertionWord`, `gapCount`; one separation per pair of consecutive runs, not one per absent slot.
Adjacency and moves	`Condorcet.adjacencyCount`, `HasLift`, `HasDrop`, `Free`, `Frozen`, `Locked`: each swap edge twice; lifts/drops may jump.
List domains	`CDCert.K6.IsDom`, `DomE`, `CDCert.ListOrder.groupsEP`: full constraint systems on all permutations, including pair choices; values on other lists are irrelevant.
Finite potentials	`CDCert.K6.cost10`, `qD`, `lamD`: respectively $10C, 2dq, 2d\Lambda$, with fixed literal data.

A.4 What is not an additional formal theorem here

The helper facts about a particular labelled example, the arbitrary number-of-copies deletion-product formula, the informal twelve-idea classification and the discovery narrative are written explanations. They are not advertised as separately exported Lean theorems. The formal amplification instead uses the proved adjacency product and powers of two. The original project's arbitrary-real-density transfer has no separate parameterised declaration; the proved 3/8 instance is exactly the one this tutorial uses.

There are 326 project source modules, with 319 in the dependency closure of the two final entry modules. Seven auxiliary modules lie outside that closure: `Audit`, `CDCert`, `CDCert.Levels`, `ClaudeAudit`, `Condorcet`, `KernelTransfer`, `ThreeOrders`. They are supplied and were compiled in the earlier complete audit. They are not extra assumptions or missing entry points of the final theorem. Generated finite-family theorems and proof-engineering helpers are included as dependencies; their precise statements can be inspected in the complete source.

B Verification evidence, reproduction and scope

B.1 What was checked, and when

The formal proof is the checked joint project of 29 October 2026. The repository's root directory `lean/` contains the complete project: all 326 source modules, `lean-toolchain`, `lakefile.toml` and `lake-manifest.json`. Every project source hash matches the earlier successful independent source build. The new statement map was made by inspecting the definitions and actual theorem declarations, and comparing them with the preserved compiler-printed final types, rather than relying on comments alone.

The earlier source build compiled all 326 project modules independently, including the 319-module closure of `CDCert.Main` and `CDCert.TwoInputs`. The earlier 20-result axiom inspection passed with only `propext`, `Classical.choice` and `Quot.sound`. These are standard Lean foundations. There is no `sorry`, project-specific axiom or `native_decide` in the checked source project. In particular the

final theorem does not assume the finite inequality, the free-to-locked injection, or the correctness of a native program.

The fresh correspondence check on 1 November 2026 succeeded. `audit/TutorialCorrespondence.lean` checks 23 explicit restatements: the three semantic interfaces and all twenty audited results. `audit/TutorialSupportStatements.lean` checks 31 further restatements, covering completion, deletion, fibres, moves, components, concatenation, the base values and the exact rational form of the finite inequality. All 54 compile without added assumptions. `audit/TutorialNamedInterfaces.lean` checks all 190 original definitions and helper declarations named in the proof, prints the bodies of the 72 definitions, and checks every axiom list. Only the standard foundations listed above occur.

These correspondence checks were compiled against hash-verified proof objects from the earlier complete source build. Their successful logs, wrapper sources and axiom lists are supplied with the tutorial. This expository revision preserves the original formal proof and does not claim another complete Lean source build.

The files `audit/SOURCE_IDENTITY.json`, `audit/TUTORIAL_LEAN_MAP.json`, `audit/ENTRYPOINT_SIGNATURES.txt` and `audit/prior_kernel_build/AXIOM_CHECK_RECEIPT.json` record the original evidence. `audit/FORMAL_STATEMENT_COVERAGE.json` maps every new restatement to its tutorial statement or definition. The successful current logs, source hashes and axiom lists are recorded in `audit/fresh_check/CORRESPONDENCE_VERIFICATION_RECEIPT.json`. The corresponding human proofs, finite normalisations and named helper interfaces are given beside each statement in this tutorial.

B.2 Rebuilding the formal proof

Use Lean 4.26.0 and Mathlib v4.26.0 with the transitive revisions pinned in the manifest. From the repository root:

```
cd lean
lake exe cache get
lake build CDCert.Main CDCert.TwoInputs
lake env lean receipts/FinalAxioms.lean
```

The cache command obtains compiled standard libraries. The build command then checks the project sources. Its large finite kernel computations may take substantial time and memory. A successful result requires completion, not a partially built directory or an import that timed out. To check the additional correspondence source once that build is available, run from the same `lean/` directory:

```
lake env lean ../tutorial/audit/TutorialCorrespondence.lean
lake env lean ../tutorial/audit/TutorialSupportStatements.lean
lake env lean ../tutorial/audit/TutorialNamedInterfaces.lean
```

The complete formal proof is independent of executing the native Python/C++ generators. The repository also contains the native generators, exact data and separately dated replay/regeneration records. They are not needed as trust assumptions of the Lean theorem.

B.3 Written proof versus a literal formal statement

A formal proof certifies a precise statement relative to its definitions. Here the semantic definitions quantify over actual strict total orders and every odd voter profile. The order/list equivalence, correspondence of laws and fixed pairs, and preservation of cardinality and swaps are proved inside the project. The tutorial's real and rational notation is matched to the natural/integer statements by the explicit index and scaling tables. These are the substantive safeguards against proving a bound for a different class or a differently normalised statistic.

A human proof may use equivalent steps: the prose amplification uses arbitrary t and a limit; Lean uses powers of two and an integer contradiction. The hypotheses and conclusion agree. The correspondence paragraphs explain such differences.

The following remain outside the literal theorem coverage: intuitive commentary, exact labelled examples checked by a separate small script, search history and performance, bibliographical claims, attribution and novelty. The twelve-idea division is an organisational choice, not a formal theorem. No priority claim follows from successful formalisation. The numerical comparison tables are exact integer calculations from the proved recurrences, with long displayed entries rounded upwards. The optional classified starting values are explicitly external inputs, and no additional Lean theorem is claimed for them or for the individual table entries. The new examples and tables are checked separately by `scripts/build_numerical_examples.py`.

B.4 Sources and credit

The mathematical source is the joint paper by P. Moros and S. Riis, *Improved Upper Bounds on the Size of Condorcet Domains* [1], now available as the review draft of 1 November 2026 in the accompanying repository. Its earlier source and the unchanged formal proof are preserved in the checked snapshot of 29 October. Moros's earlier note [2] is an AI-assisted precursor within the same investigation. The joint paper presents the improved 32/11 bound and its component reduction, together with the peak-pit bound 8/3.

This tutorial is an exposition by Søren Riis of that joint work. Claude (Anthropic) and ChatGPT/Codex (OpenAI) assisted the exploration, checking and drafting of both the research and explanatory material. Computational optimisation selected finite correction coefficients; their validity is established by exact verification. The tutorial's byline identifies responsibility for this exposition, while the underlying results and formalisation remain credited to their sources. The sources also credit Fathi's counting observations and the majority/median background of Puppe and Slinko.

The following background references orient the general mathematical ideas; the proof above and its explicit certificate provide the actual logical ingredients for this tutorial.

References

[1] P. Moros and S. Riis, *Improved Upper Bounds on the Size of Condorcet Domains*, joint review draft, 1 November 2026; complete accompanying Lean formalisation and computational material. `https://github.com/SR123/condorcet-upper-bound`. The formal proof is pinned to snapshot `0d5dd168854b` (29 October 2026).

[2] P. Moros, *Condorcet domains have fewer than* 2.95^n *orders*, earlier AI-assisted research note (2026). `https://doi.org/10.5281/zenodo.23654773`.

[3] A. K. Sen, A possibility theorem on majority decisions, *Econometrica* 34 (1966), 491–499.

[4] C. Puppe and A. Slinko, Condorcet domains, median graphs and the single-crossing property, *Economic Theory* 67 (2019), 285–318. `https://doi.org/10.1007/s00199-017-1084-6`.

[5] D. W. Cranston and D. B. West, An introduction to the discharging method via graph coloring, *Discrete Mathematics* 340 (2017), 766–793. `https://arxiv.org/abs/1306.4434`.

[6] K. K. H. Cheung, A. Gleixner and D. E. Steffy, Verifying integer programming results, *IPCO 2017*, 148–160. `https://arxiv.org/abs/1611.08832`.

[7] D. Akello-Egwell, C. Leedham-Green, A. Litterick, K. Markström and S. Riis, *Condorcet Domains of Degree at most Seven*, arXiv:2306.15993, version 5 (2023), Section 4.1 and Tables 1–4. `https://arxiv.org/abs/2306.15993`.

[8] C. Leedham-Green, K. Markström and S. Riis, The largest Condorcet domain on 8 alternatives, *Social Choice and Welfare* 62 (2024), 109–116, Theorem 4.1. `https://doi.org/10.1007/s00355-023-01481-3`.